

CPU Simulator - A Learning Tool

This document accompanies a Java project (CPU simulator) that can be used to learn-by-doing. However, you will learn about all kinds of CPUs and various types of assemblers, not just the project CPU. That project can be found at <https://github.com/Jim-Clack/CPU>

1 - Building Blocks of a Computer

Step 1 is to create the building blocks of the computer, a Bit, a Byte, and a LogicGate.

Bit

We aren't worried about efficiency or performance, as we are trying only to simulate the way the hardware works. So we'll store a bit in the lowest bit of an integer. Keep in mind that you must ensure that it is always 0 or 1 when you set `val`. There are three ways of doing this, either (a) use the & and operator, or (b) use the % modulus/remainder operator, or (c) simply convert any non-zero value to a 1. Here is a look at the Bit class that you need to write. BTW - Each of these methods can easily be implemented in a single statement - one line of code.

```
public class Bit {
    private int val = 0;
    public Bit(Bit bit) ...
    public Bit(int val) ...
    public Bit(boolean val) ...
    public int getVal() ...
    public Bit inverse() ...
}
```

BByte

We will use the term BByte instead of Byte because Java reserves the keyword Byte. IMPORTANT: The bits in a byte (or BByte) are numbered from right-to-left, with the least-significant bit being referred to as bit number zero. This is because of the numeric nature of a byte. When you are working with regular (decimal) numbers, the *units* position is on the right. Therefore, the binary *units* position in a byte is also on the right. Make sure you implement this correctly in `setBit()` and `getBit()` in your code.

Bit-Numbering Reference								
7	6	5	4	3	2	1	0	⇐ Bit Number
128	64	32	16	8	4	2	1	⇐ Bit Weight

```
public class BByte {
    public static int NumBits = 8;
    public static int MaxBitNum = 7;
    public static int MaxBitWgt = 128;
    public static int NextBitWgt = 256;

    private final Bit[] content = new Bit[NumBits];

    public BByte() ...
    public void set(BByte from) ...
}
```

```

public void setBit(int bitNum, Bit bit) ...
public Bit getBit(int bitNum) ...
public void set(int from) ...
public String toString() ...

```

Hint:

You'll need to make use of shift operators and logical (&) operators. And remember to create a new Bit each assignment, not just set the current bit to a new value. Why? Because, if you don't instantiate a new Bit, Java will allow multiple references to the same object - the same Bit - and if code were to change the value of one of them, it would change the other as well. For instance:

```

public void set(int from) {
    for (int i = 0; i < NumBits; i++) {
        setBit(i, new Bit((from & (1 << i)) > 0));
    }
}

```

More BByte Methods

There is no need to implement the following BByte methods yet. But, as we move forward, you will have to add these as well, in order to support other needs. These will make use of bit movement (shift), bit-tests (0 versus 1), and logic (we will discuss LogicGates next).

Note that some methods will involve basic arithmetic, but the computer cannot (yet) perform arithmetic, so how could these be implemented in hardware? The answer is two-fold: (a) Those methods are only used for getting data into and out of the computer so they are not needed for basic logic and program flow, and (b) You will soon see how arithmetic can be implemented in hardware by combining LogicGates to create a HalfAdder and an Adder.

So, if you feel that you are ready to do so, go ahead and start implementing some of these BByte methods.

```

public BByte(String bits) ...
public BByte(int intVal) ...
public void set(BByte from) ...
public void set(String from) ...
public int getSignedValue() ...
public int getUnsignedValue() ...
public Bit isZero() ...
public BByte onesCompliment() ...
public Bit shiftLeft(int howFar) ...
public Bit shiftRight(int howFar) ...

```

LogicGate

Since you are going to learn about logic gates and are not yet ready to implement them yourself, here is the entire source code for four kinds of LogicGates. Notice that they are implemented as a single enumeration. We will discuss these and make use of them as *the basic building blocks for any computer*.

```

public enum LogicGate {
    AND("0001"),
    OR("0111"),
    XOR("0110"),
    NAND("1110");
}

```

```

private final Bit[] truthTable = new Bit[4];

LogicGate(String truthTable) {
    for(int i = 0; i < truthTable.length(); i++) {
        this.truthTable[i] = new Bit(truthTable.charAt(i) % 2);
    }
}

public Bit gate(Bit a, Bit b) {
    return truthTable[(a.getVal() * 2) + b.getVal()];
}

@Override
public String toString() {
    return this.name() + "[" + truthTable[0] + ", " +
        truthTable[1] + ", " + truthTable[2] + ", " + truthTable[3] + "]";
}
}

```

2 - Understanding Basic Operation

The building blocks in the previous section are *everything* that comprises a computer except for I/O. You will soon see how these things can be combined in order to do addition, subtraction, multiplication, division, conditionals, loops, functions, and so forth. Once you understand what a Bit and Byte really are, and what the LogicGates do, you will be knowledgeable about digital electronics.

Digital Electronics

A computer is built up from transistors. A transistor is like a valve or a faucet for electrons to flow through. Each transistor has an input, an output, and an emitter/gate that controls how much electricity can flow through it. Multiple transistors can be wired together in such a way to create a bit, a byte, or a logic gate. And that is all you need. Well, except for one minor detail: the clock.

In every computer there is a constant ticking of a clock, often referred to as an M1 cycle. When you mention your computer's clock speed, this is the clock you're referring to, not the time-of-day clock. In a computer there are many things going on at the same time, just like a multi-threaded application in software. And, similarly, there is a potential problem with synchronization. Imagine that you have to make a decision based on two inputs but they may not arrive at exactly the same time. As you might imagine, this is a problem.

So everything that occurs in the hardware is synchronized by the clock. The electronic engineers have designed the circuitry based on how long it takes for each event to occur and then determined the clock speed based on that. The way it works is that none of the LogicGates can gate - make a decision to determine the output - until they get a clock signal. That signal is said to "latch the output", which is to make the expected decision then "lock it in" so it doesn't change until the next clock cycle.

Many of the LogicGates in the computer are implemented via flip-flops. A flip-flop is simply a gate that accepts a (clock) signal that will latch its output until it gets another signal. If the flip-flop is involved in program flow, then the M1 clock cycle is expected to latch it.

That is digital electronics in a nutshell. Soon, you'll learn how we can combine these building blocks to create an entire computer.

Logic Gates

You just saw how simple a logic gate is, as four of them were coded in a few lines of Java. A logic gate takes two binary inputs and, based on their state, generates a binary output state. For instance, with an AND gate, if two inputs are both at a state of 1 then the output of the gate is 1. But if either or both of the inputs are at a state of 0 then the output will be zero. That is why it is referred to as an "AND" gate - both input a AND input b must be at a 1 state in order to set the output state to 1.

AND Gate: Outputs a 1 if $a = 1$ AND $B = 1$, else it outputs a 0.

For an OR gate, either input a OR input b must be at a state of 1 in order to set the output state to 1. If a is 1, or if b is 1, or if both are 1, then the output is 1. Hence, the name "OR" gate.

OR Gate: Outputs a 1 if $a = 1$ OR $B = 1$, else it outputs a 0.

An XOR gate is referred to as an eXclusive OR gate because, like an OR gate, it outputs a 1 if a OR b are 1. But, unlike an OR gate, if both a and b are 1, it outputs a 0.

XOR Gate: Outputs a 1 if $a = 1$ OR $B = 1$, but not both a and b, else it outputs a 0.

A NAND gate is identical to an AND gate except that the output is inverted.

NAND Gate: Outputs a 0 if $a = 1$ AND $B = 1$, else it outputs a 1.

These are the four most common kinds of logic gate. They can be described with a Truth Table like this:

a input	b input	AND output	OR output	XOR output	NAND output
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

Everything beyond the simple wiring in a computer is performed by a variety of logic gates interconnected in various ways. Logic gates are responsible for arithmetic, program flow, function calls, and so forth.

Unit Testing

Let's do some tests to verify that the building blocks work as expected. In order to do this, you may have to implement some of the additional BByte methods described in the previous section. Here's source code for LogicGateTest and BByteTest.

```

@Test
void testLogicGates() {
    testGate(LogicGate.XOR, (a, b) -> a ^ b);
    testGate(LogicGate.AND, (a, b) -> a & b);
    testGate(LogicGate.OR, (a, b) -> a | b);
    testGate(LogicGate.NAND, (a, b) -> 1 - (a & b));
}

void testGate(LogicGate gate, IntBinaryOperator tester) {
    System.out.println("LogicGate Test: " + gate);
    for(int a = 0; a < 2; a++) {
        for (int b = 0; b < 2; b++) {
            Bit bitA = new Bit(a);
            Bit bitB = new Bit(b);
            Bit result = gate.gate(bitA, bitB);
            System.out.println(" a=" + a + ", b=" + b + ", ==>" + result);
            int expected = tester.applyAsInt(bitA.getVal(), bitB.getVal());
            assertEquals(expected, result.getVal());
        }
    }
}

@Test
void testBByte() {
    System.out.println("BByte Tests:");
    testSignedValue(86, new BByte("01010110"));
    testSignedValue(54, new BByte(0x36));
    testSignedValue(-107, new BByte(-107));
    testSignedValue(-108, new BByte(107).onesCompliment());
    testSignedValue( 107, new BByte(-108).onesCompliment());
    testSignedValue(-128, new BByte(128));
    testSignedValue(-2, new BByte(254));
    testSignedValue(-1, new BByte(255));
    testUnsignedValue( 0, new BByte(0));
    testUnsignedValue(127, new BByte(127));
    testUnsignedValue(128, new BByte(128));
    testUnsignedValue(254, new BByte(254));
    testUnsignedValue(255, new BByte(255));
    testUnsignedValue(255, new BByte(-1));
}

void testSignedValue(int expected, BByte actual) {
    System.out.println(" " +
        expected + " ==?= " + actual.getSignedValue() + " ==> " + actual);
    assertEquals(expected, actual.getSignedValue());
}

void testUnsignedValue(int expected, BByte actual) {
    System.out.println(" " +
        expected + " ==?= " + actual.getUnsignedValue() + " ==> " + actual);
    assertEquals(expected, actual.getUnsignedValue());
}

```

Make sure that these unit tests succeed before going on.

Topics for Discussion

We need to talk about these things first. Then we can build the computer simulator.

- Universal Turing machine and procedural/imperative (or OO) programming
- Lambda Calculus and functional programming
- Little endian (versus big endian) - byte order

- Logic, shift, move, test - bit operations only
- No math except on real-world conversions (strings, decimal values)
- Note: iterating over bits at a low level is really "selection" not addition
- Two's complement notation (negative values, numeric invert b boolean invert)

3 - Arithmetic and Logic

Maybe it seems too simple to you, or maybe it's overwhelming and difficult to understand. If you are not comfortable with these building blocks, do not go on. Review, experiment, research, and ask questions, until you fully understand it all - before you continue.

HalfAdder

This is another kind of logic gate that comprises two logic gates. It is called a HalfAdder because you will need two of them to add two bits together. For instance, if we need to add two bytes, eight bits each, we'd need 16 HalfAdders. In hardware, this is exactly how it's done. But in software it is easier to just use two HalfAdders per bit and then iterate over the eight bits in the bytes.

Here is the logic for a HalfAdder:

```
sum = LogicGate.XOR.gate(p, q);
carry = LogicGate.AND.gate(p, q);
```

That's all there is to it. Notice that the sum is only one Bit. If we add two Bits, they might sum up to more than one Bit, so how is that handled? That is the purpose of the carry Bit. When we stack our HalfAdders to add up Bytes instead of Bits, we will have to handle the carry when we get to the next bit to the left. We always add the bits from right-to-left. This is exactly like regular (decimal) addition and should be familiar to you. Please make sure you understand this:

How to "carry" the overflow from one bit/digit to the next one to the left:

	11		1 1	← carry
Decimal	168	Binary	10101000	
Addition	+ 45	Addition	+ 00101101	
	-----		-----	
	213		11010101	

Adder

How do we use 2 HalfAdders to make a full Adder?

- First of all, an Adder handles the carry to the next bit/digit, so it will always involve more than one Bit. Therefore, we will build our Adder by extending an BByte. The BByte that is used by the Adder is often referred to as an accumulator.
- Secondly, we'll have to write code that iterates over the bits, beginning with the rightmost bit, Bit 0, then incrementing to the left. The addition of each bit will involve two HalfAdders, one to add the bits and one to add in the carry. The first bit, the rightmost bit, Bit 0, will start with an incoming carry of 0, of course, because there are no bits to its right.

- Finally, we have to remember to keep track of the carry from each iteration so we can add it into the next iteration - the next Bit to the left. And remember: The right-most bit is Bit number 0.

Once you understand these three principles, you should be able to see how an Adder works:

```

    carry = new Bit(0);
    for(int bitNum = 0; bitNum < BByte.NumBits; bitNum++) {
        halfAdder.add(this.getBit(bitNum), bbyte.getBit(bitNum)); // add bits
        Bit step1Carry = halfAdder.getCarry();
        halfAdder.add(halfAdder.getSum(), carry); // add carry
        carry = LogicGate.OR.gate(halfAdder.getCarry(), step1Carry);
        this.setBit(bitNum, halfAdder.getSum());
    }

```

That's it. This was the most complicated part of digital electronics. You now know everything you need to know to create an entire computer. Most likely, though, you do not yet know how to put all the pieces together.

See if you can write the code for the HalfAdder and the Adder. Here are the skeletons for you to fill out.

```

public class HalfAdder {
    private Bit sum = new Bit(0);
    private Bit carry = new Bit(0);
    public void add(Bit p, Bit q) {
        ... // ← You were told (see HalfAdder, above) what goes here
    }
    public Bit getSum() { return sum; }
    public Bit getCarry() { return carry; }
}

public class Adder extends BByte {
    protected Bit carry = new Bit(0);
    public BByte add(BByte bbyte) {
        HalfAdder halfAdder = new HalfAdder();
        carry = new Bit(0);
        ... // ← You were told (see Adder, above) what goes here
        carry = halfAdder.getCarry();
        return this;
    }
}

```

More Complex Operations

- Transistors in most computers are CMOS circuits built on a semiconductor Substrate.
- LogicGates are actually built from those Transistors.
- FlipFlops are built by combining LogicGates.
- Memory, HalfAdders, and Adders are made up from FlipFlops and LogicGates.

As you can see, we now have the ability to move data, Bytes and Bits, to perform Logical operations on it, and to do Addition on positive integers. What else is there?

- Handle negative integers?
- Perform subtraction?
- Compare for equality, less-than, and greater-than?

Believe it or not, those are freebies. Your code already handles all of those things. Really. We will soon discuss two's-compliment arithmetic, which is just a way of using our existing code to do all three of those capabilities with minimal effort. Yes, but how about...

- Multiplication?
- Division?
- Floating point numbers?

Simple: Those aren't handled directly by the hardware. The computer has subroutines, software stored as firmware within the CPU called "microcode" that does long-division and handles floating point numbers the same way you do on paper, multiple steps with simple integer operations. So it can all be done with an Adder. Of course, some CPUs have additional gates set up to perform common functions that will be called by the microcode, but the Adder is the key.

Two's-Compliment Arithmetic

Negative numbers, subtraction, and comparison can all be performed by the Adder that you just created. Let's begin with negative numbers.

We will first discuss this approach using regular decimal numbers - ten's-compliment notation. Imagine that we have only two digits so we can count from 0 to 99, but we need to count from -50 to +49. We need negative values as well as positive values. All we need to do is to give certain numbers an alias. We can still use 0 to 49 as our positive integers. But we will use the values 50 to 99 to represent the negative values -50 to -1. So 97 is the same as -3, and 95 is the same as -5, for instance. How does this work?

Let's try adding -2 and -3. These two values will be represented by 98 and 97, respectively. When we add 98 and 97, we get 195. Of course that is too large to fit into our two digits, so the leading 1 is lost and we end up with 95. Notice that the alias for -5 is indeed 95. It worked!

As you can see from this example, we can use the values from 0 to 99 to represent an unsigned value from 0 to 99 or a signed value from 0 to 49 and -50 to -1. Think of that. The same math works on signed or unsigned values. All we need to do is to cover negative values to their appropriate value when we input them or output them. The exact same math works either way.

Ten's-Compliment and Two's Compliment Notation

Analogy to Two's-Compliment 8-Bits, but using Ten's-Compliment 2-Digits instead:

- For unsigned values $\Rightarrow$ the range is 0-99.
- For signed values $\Rightarrow$ values ≥ 50 will be assumed to be negative.

Addition Example

Unsigned	Signed
97	-3
98	-2
---	---
95=Sum (Carry=1)	-5
95 == -5	96 == -4
97 == -3	98 == -2
	99 == -1

Example...

```
set(95) -> 95 (95 is simply the value 95)
set(-5) -> 95 (-5 is represented by 95 for a signed value)
```

Also, note that we can subtract a value (95) from the maximum possible value (99) then add 1 to negate a value.

Now, let's negate a Two's-Compliment 8-Bit value. Similarly, we subtract it from the maximum value (255), which is the same as inverting all the bits, then adding 1:

```
95 =                                01011111
inverted (one's compliment) =>      10100000
+1 add one =>                        10100001
```

The result is 161 unsigned but is also -95 as a signed value.

Thus, we can `set()`, `add()`, `subtract()`, and otherwise process values without concern for whether they are positive or negative, signed or unsigned. The only difference between them is how we interpret them in the getters:

```
number.getUnsigned(); // if bit 7 == 1, it's at least 128
number.getSigned();   // if bit 7 == 1, it's negative
```

We can do the same thing with binary numbers, using Two's-Compliment notation instead of Ten's-Compliment. For one byte, 8 bits, we can use it to hold unsigned (positive) values from 0 to 255, or signed (+/-) values from -128 to 127. Either way the values and the arithmetic are identical - the only difference is how we interpret the values. In other words, when we retrieve a value, we can interpret it as signed or unsigned - hence `getSignedValue()` and `getUnsignedValue()`.

Negation

A benefit of two's-compliment notation is that a number can be negated very simply, by inverting all the bits then incrementing it. Inverting the bits, converting zeros to ones and ones to zeros, is referred to as one's-compliment or logical inversion.

Consider the binary value of 23:

```
00010111
```

Invert the bits:

```
11101000
```

Then increment the byte:

```
11101001
```

As expected, this represents -23 in two's-compliment notation. Notice that this same approach can be used to negate the -23 back to a positive 23.

Subtraction

Since we know how to negate a value, per the previous topic, we can perform subtraction. We simply negate the value to be subtracted then add it.

Floating Point

Of course, much of computing involves scientific or technological values with a decimal point. As you learned in third or fourth grade, you can perform floating point arithmetic using integer operations, as long as you follow certain steps - an algorithm. Similarly, a computer uses integer arithmetic and some highly sophisticated algorithms to perform floating point arithmetic. In small computers, this is done in software - code libraries. In larger computers, it is done in microcode, code that is embedded into the CPU. Advanced computers have certain steps from some of these algorithms implemented in the hardware.

There is a standard for floating point values, IEEE 754, that facilitates all of the necessary mathematical operations and supports various lengths and precisions of numbers. So all modern general purpose computers represent integers in two's-compliment form and floating point numbers in IEEE 754 form. Generally, you won't need to know how to read or write floating point values in binary. Just in case you want to know how, here's a Quick-Ref:

How an IEEE 754 Floating-Point Number is Represented

Notes: 1-Sign bit, Seven 1's are special cases, Exponent is in excess-128 notation. Significand has an assumed leading 1, which is omitted from the bits.

Example: S Exces128 FractionPrefixedAssume1
 3 32222222 2221111111111000000000
 1 09876543 21098765432109876543210
 if these ^^^^^^ 7 bits are 1's, it's a special case of One/NaN/Inf

Breakdown of a floating point value...

1. If the 7-V bits are set, it's a special case of One/Nan/Infin, else:
2. The leftmost bit becomes the negative sign (0=positive, 1=negative)
3. Take the next 8 bits, subtract 128 (10000000), and that is E (signed)
4. Take the remaining bits, prepend a "1", and that is the mantissa in binary

Flag Bits

After each arithmetic or logical operation, flag bits are set. These are one-bit values that can be read by the program. Often they are kept in a register for easy access. There are always at least three flag bits:

- **Zero Flag** - This indicates that the result (accumulator) is all zeros. In other words, if the result of the operation is a zero byte, then the flag is set to one. This can be confusing to noobs, as one indicates zero, and zero indicates one. But it makes sense when you understand that "true" equals 1 and the flag indicates zero.
- **Carry/Overflow Flag** - This is set only if the previous operation tried to carry over past the left of the leftmost bit, or in certain operations, past the rightmost bit. In other words, it means that the operation overflowed, so a bit of data was lost, generally because there was an outstanding "carry" when the Adders finished.
- **Sign/Neg Flag** - This confirms that the leftmost bit of the result is a one. For signed two's-compliment values, this indicates that the result is negative. Refer back to the two's compliment topic and examples and you'll see that negative values always have the leftmost bit set.

Comparison

Whether we are comparing integers, floating point numbers, strings, or anything else, the comparison is always numeric and reduced to an integer comparison. For instance, to compare strings to place them in alphabetic order, we compare the numeric values of each character.

Comparing values is the same as subtracting them from each other, discarding the result, then checking the flags, as follows:

- If the Zero Flag is set, then the values are equal. (Because the result of the subtraction was zero.)
- If the Carry/Overflow Flag is set, then the subtracted value is larger.
- If neither of those Flags are set, then the subtracted value is smaller.

Summary: Addition, subtraction, negation, and comparison are all performed by an Adder.

Registers

Whereas memory is simply an array of bytes (BBytes), in most cases, the registers are far more capable. Registers contain two's-complement values and are affiliated with Adders so that arithmetic and logic can be used with them. In rare cases, an Adder can be affiliated with regular memory, but it is typically implemented by first loading that memory cell into a register within the CPU. Registers are far faster than regular memory because they are inside of the CPU along with all the logic. But there are only a few registers.

On CISC computers, like the PC/MAC/Pentium, most registers are special-purpose. One is optimized as a counter, another is optimized for use as an indexer to point to elements of an array, and yet another is dedicated to use as a stack pointer. These registers have names that suit their purpose, for instance: SP = Stack Pointer, EIX = extended indexer, etc.

On RISC computers, such as a Smartphone/ARM, most registers are general-purpose and can be used as you wish, although certain registers are defaulted to certain purposes. These registers are most often numbered, like memory: register 1, register 2, and so forth.

Key Takeaways: (IMPORTANT)

1. Integer arithmetic and logical operations need not be concerned with whether a value is negative or positive because the process is the same either way.
2. But an integer getter is where concern is necessary. There must be two getters, one for unsigned values and another for signed values. Each one processes the values differently. Most programming languages take care of this for you by requiring you to specify, in the declaration or cast, whether an expected value is signed or unsigned. In Java, the default is always "signed."

4 - Opcodes and Arguments

The only significant functionality we have not yet discussed is program flow: conditionals, loops, Fdas

Opcodes

On CISC computers, like the PC/MAC/Pentium, there are hundreds or thousands of opcodes, where many are just minor variations and some are specialized instructions for optimizing performance. On RISC computers, such as a Smartphone/ARM, there are usually only a hundred or fewer opcodes, which allows them to be shorter and for the compiled code to take up less memory and run faster. Each approach has its own benefits and drawbacks.

Here are some typical opcode mnemonics:

- LOAD - Take a value from memory or somewhere else and put it into a register.
- STORE - Take a value from a register and put it into memory or somewhere else.
- PUSH - Put a value onto the stack, where multiple values can accumulate in LIFO order.
- POP - Remove a value from the stack and put it into a register or, possibly, elsewhere.
- ADD - Add a value (from memory or another register) to the specified register.
- JMP - Change in program flow, like a goto statement, that resumes execution elsewhere.
- CALL - Push the program counter and jump to a subroutine, function, or method.
- RET - Pop the program counter, returning from a subroutine, function, or method.

Flags and Conditions

Some operations are conditional. Almost every assembler allows the programmer to append these to certain instructions, generally the JUMP opcodes, to make them execute only if a certain condition has been met. These conditions are kept in a FLAGS register and are typically only set by arithmetic or logical operations. In other words, if you LOAD, STORE, or POP a register, the condition flags will not be adjusted. But if you do an ADD, CMP, or an XOR, then the conditions in the FLAGS register will be set according to the result of that operation.

For instance, if you want your code to jump to the Exit: label if the last ADD operation had a result of zero, you would code:

```
JZE Exit:
```

This means “Jump, if the result was zero or equal,” to the Exit label.”

You may wonder why we say “zero or equal.” The answer is due to two’s complement arithmetic. Remember that subtraction (SUB) is a form of addition (ADD) and that a comparison (CMP) instruction is merely a subtraction that discards the result. Look at these three operations and consider how they deal with two’s complement numbers.

```
ADD 77 + (-77)    ...or...  
SUB 77 - 77       ...or...  
CMP 77, 77
```

These are all the same operation with different pre- or post-processing. In the case of ADD or SUB, the result is “Zero.” But in the case of CMP, the result is discarded, so the result is “Equal.” Therefore, “Zero” and “Equal” are the same flag - the ZE flag. Here are the three most commonly used flags:

ZE	Zero or Equal	(ZERO flag is 1)
GT	Greater Than	(SIGN flag is 1)
LT	Less Than	(ZERO and SIGN flags are 0)

And here are their inverses - opposite in meaning:

NZE	Not Zero or Equal	(ZERO flag is 0)
LTE	Less Than or Equal	(SIGN flag is 0)
GTE	Greater Than or Equal	(ZERO and SIGN flags are 1)

These are the six conditions, based on the flags, that you need to know in order to perform conditionals, loops, and other program flow control.

Typically there is also a carry (CY) flag that indicates that a bit was lost, that it fell out of the end of the register during the last arithmetic or logic operation. And there is usually a flag to indicate whether interrupts are enabled or not.

Arguments and Address Modes

In the previous topic, you may have noticed that data can be moved from/to a variety of places, such as a register, a memory cell, a value in the code, or the stack. How does it know where? The answer is that this is specified in the coded instruction - the assembled opcode - machine language. For instance, there may be several variations of the ADD instruction, where one of them adds a value from memory and another one adds a value from another register. This is referred to as the "address mode."

But different assemblers have different syntax, even on the same computer. You need to refer to the documentation for each assembler in order to understand how this is done. With some assemblers, you have to add a suffix to the opcode to indicate the address mode. Other assemblers use punctuation on the arguments to do the same thing. For instance, here are examples on three different assemblers:

Opcode Instruction	Assembler A	Assembler B	Assembler C
Register 1 + Memory[1234]	ADDMEM 1,1234	ADD #1,[1234]	ADD REG1,1234
Register 1 + 1234	ADDIMM 1,1234	ADD #1,1234	ADD REG1,'1234'
Register 1 + Register 2	ADDREG 1,2	ADD #1,#2	ADD REG1,REG2
Register 1 + Local Variable 3	ADDLRA 1,3	ADD #1,[BP+3]	ADD REG1,(FP+3)
Register 1 + Memory[Register 7]	ADDIND 1,7	ADD #1,[#7]	ADD REG1,(REG7)

(For the example project that accompanies the document, the first approach was used, merely because it is a "mini" assembler and this is the simplest to code.)

Common Address Modes:

- Register - Argument is a register that contains the value.
- Immediate - Argument is a constant - the value.
- Memory/Direct - Argument is in a memory cell with the value.
- Indirect - Argument is a register (or mem) that points to the memory cell with the value.
- Indexed - Argument is an offset from a register, to a memory cell with the value.
- Local/Frame - Argument is an offset into the frame, to the memory cell with the value.

So the address mode may be specified via a suffix on the opcode or by certain punctuation on the operands, depending on the assembler. In the examples above, note that the first argument is typically the destination. This is generally true for all instructions other than the STORE opcodes. However, some assemblers replace LOAD and STORE with a MOVE instruction and it handles both the load and store functionality by requiring that the destination is always the first argument. In other words, when the register is the second argument, it becomes a STORE instruction.

The project associated with this document (CPU) uses the first approach (Assembler A, above) so the following instruction is a valid mnemonic. Can you figure out what it does?

JLTIEM Loop:

You may wonder why we used address mode IMM (Immediate) and not MEM. (Memory) The reason is that we want to jump to the label at that memory cell, which is a constant. If we were able to use “**JLTMEM Loop:**” then we would be telling it to fetch the *integer value* from that memory cell and use it as a pointer, telling it where to jump to. The MEM address mode refers to the content (value) of that memory location, not the memory location itself. This is often a point of confusion for new assembly language programmers.

Note that there are some computers, referred to as “two address” machines, that allow you to apply an address mode to both arguments. But most computers only allow it on the last argument.

Functions, Methods, Subroutines

Fdasf

Global Data and the Heap

fdas

Objects and “this”

Fda

5 - Program Execution

The only significant functionality we have not yet discussed is program flow: conditionals, loops, and function calls.

Instruction Pointer and Jumps

Dasdas

Conditionals and Loops

fdas

Stack Pointer

Gfagf

Calls and Returns

fdaS

Frame Pointer and the Stack Frame

Das

Enter and Leave Operations

Fdasfd

dfsgafgfa
fdasfda

6 - Input and Output

I/O Ports

Fdasfd

Interrupts

Hjk

ISRs and Handling Interrupts

Ghkj

Device Drivers

Normally, a device driver does not only do input or output, but both. For instance, in order to send output to a console, the driver must first ask the console about its capabilities, resolution, and so forth. Before it sends data to it, it has to ask if it's ready to receive. So, whether your device is input or output, it usually both sends and receives.

Connection + Four System Coding Areas

The connection to a device involves cabling as well I/O ports or memory-slot I/O, or direct memory access, as well as physical connections for input, output, and the generation of interrupts. Typically, this is all incorporated into an adaptor, such as a network interface card (NIC), a disk-drive interface (SATA), or a USB adaptor on a chip, which is typically mounted on the motherboard. Moreover, there are four system design areas: Driver Support, Driver API, Driver ISR, and Interface Card Firmware. Often, the peripheral manufacturer is expected to provide all of these things to support their hardware.

(1) The Driver API is simply the code that gets called by the operating system when an application wants to communicate with the device. Operating systems prevent the application program from accessing a device directly, as that endangers all apps on the computer and risks data loss or corruption. So, your code uses a File, a Socket, a Stream, or some other object to tell the operating system what it wants to do. Then the operating system calls the Driver API to do it. One aspect of systems programming is to write Driver APIs, often in C or C++. But the Driver API cannot communicate directly with the device.

(2) Another aspect of systems programming is to write a driver interrupt service routine, an ISR. This is the code that actually reads and writes the data. Typically, it is invoked by an interrupt when the device is ready - when it is sending to the computer's input or when it is ready to accept output. It runs in supervisor mode (privileged or ring zero) so that it is not prohibited from accessing I/O or memory directly. The programmer has to use a bit of assembler as well as C or C++ and have a good understanding of virtual memory and other OS-specific issues. Although there are a lot of rules to follow, it is not particularly difficult to code, but it must be very fast, and it cannot contain any bugs or it will crash the computer. (Blue Screen of Death) The Driver API and the Driver ISR are bundled into one file that must be carefully installed into the operating system

(3). Optionally, there may be other (higher-level) "drivers" as well, although a systems programmer would shudder to hear us refer to them as "drivers." These are software adaptors to present the nature of the Driver API in a manner that is expected by the operating system and code libraries. For instance, for a network, these might implement protocols for ARP, BOOTP, or DHCP. For an SSD, these "drivers" might serve to make the SSD appear to the operating system to be a spinning disk. For a USB printer, they might monitor ink supply or perform color compensation.

(4) And then there is the firmware on the interface card, often written in assembler. It is software that is burned into Flash or EPROM memory on that card. In the accompanying project (CPU) you will see that there is a device simulator, written in Java, that provides this functionality so that the CPU simulator can access the console - the keyboard and text display of your computer. The other system programs, above, are written in CPU-SIM Assembler.

Example Device Driver

Hjk

Disk Allocation and File Allocation

Hkl

Network Stack

Jlk

Jol